# #22 【AI投資戦略】「爆益チャート」をAIで集めたら勝率も爆上がりする？

https://www.youtube.com/watch?v=eh9d_cRocc4 ／ 1:00:34

### 3行要約

1. AI投資企画の第2話。「チャートの形（ダブルボトムなど）を、AIに画像として見せて判定させ、売買の合図にする」仕組みを、2日がかりで作る回です（本人も「過去いちばん難しい回」と言っています）。
2. Claude Codeで、ビットコインの値動きグラフを描いて切り取る道具を作り、GeminiのAPIで「この画像はダブルボトムか（1か0で）」と判定させ、過去データで成績を確かめます。
3. 結果は「ほぼ損得なし」。判定基準がゆるすぎたので、条件を厳しくして次回へ持ち越しとなりました。

### 主なポイント

- 🔑 **AIを使えば、「試す回数」が圧倒的に増える。** 以前は1つのアイデアを確かめるのに1か月かかったものが、今は数分〜数時間。これがAIを使う人と使わない人の大きな差になる（投資以外の仕事でも同じ）。
- 🔑 **チャットAIに「勝てる方法を教えて」と聞くだけでなく、「大量に検証させる」という発想を持つ。** これはClaude Codeのような道具を知っていてこそ出てくる発想（相方の気づき）。
- 時間がかかりそうな処理は、先に「進み具合を表示して」とAIに頼んでおく。
- 判定に使う指示文（プロンプト）は、結果がおかしいときに必ず中身を確認する。AIが何を基準に判断したかを一覧にさせると、問題点が見つかる。
- 無料のAPIは1分あたりの回数制限がすぐ来るので、本格的に使うなら支払い設定が必要。APIキーは「漏れても被害が小さい」管理を前提にする（→#19）。
- 実際の売買に使うなら、ネット経由のAIより自分のパソコンで動くモデル（ローカルLLM）のほうが良い。サービス側で黙ってモデルが変わり、判断がぶれることがあるため。
- 自分のやり方を言葉にするだけで、感情的な売買が減り、勝てるようになった人もいる、という話。

## 紹介されたテクニック

- **自分で「判定用の画像」を作らせる**：ネットのチャート画像を拾うのではなく、Claude Codeに「ビットコインの4時間ごとの値動きデータを取ってきて、グラフにして」と頼み、毎回同じ見た目の画像を自分で作る。最終的に自動で判定させるなら、画像の見た目が毎回そろっていないと判定がぶれるため。
- **欲しい機能はその場で言葉で足す**：グラフ画面に「四角で囲んだ範囲を切り取って、正方形で保存できるボタンを付けて」と頼むだけで、切り取り機能が1分ほどで追加された。「こういうボタンがあれば楽」と思ったらすぐ頼むのがコツ。
- **「ultrathink」で深く考えさせる**：Claude Codeへの指示の最後に「ultrathink（ウルトラシンク）」と書くと、より深く考えてから作業してくれる。複雑な仕組みを作るときに使う。
- **画像判定は「1か0」で答えさせる**：GeminiのAPI（プログラムからAIを呼び出す窓口）に画像を渡し、「ダブルボトム（底が2つある形）なら1、違えば0で返して」と指示する。答えを数字に固定すると、プログラムで「1なら買い」のように自動処理しやすい。安くて画像認識が得意な Gemini 2.5 Flash を使った。
- **やり方が分からない手順もAIに説明書を作らせる**：「GeminiのAPIキー（利用のための合言葉）の取り方を、URL付きでMarkdownファイルにまとめて」と頼み、その手順どおりにGoogle AI Studioでキーを発行した。
- **APIキーは「漏れても被害が小さい」前提で管理する**：キーは.envファイル（秘密の設定を置くファイル）に入れ、使い終わったら削除する。漏れても損害が1万〜10万円で済むよう上限を設定しておくのが安全。
- **時間がかかる処理は先に「進み具合を表示して」と頼む**：大量の画像を判定させる前に、「何件中何件終わったか、判定結果の1か0も表示して」と指示しておく。待ち時間に止まっているのか進んでいるのか分かり、詰まりにすぐ気づける。
- **まず少量で試してから本番を回す**：いきなり1,000件ではなく50件程度で動かして確認する。無料のAPIは1分あたりの回数制限にすぐ当たるので、本格的に回すなら支払い設定を入れる。
- **結果がおかしいときはプロンプトと判定基準を見せてもらう**：「使ったプロンプトと、画像ごとの判定結果を一覧にしたHTML（見やすいWebページ）を出して」と頼むと、基準がゆるすぎてダブルボトムでない形まで1にしていたことが分かった。その後、条件を厳しくする指示（大きめの形で、ネックラインを抜けた瞬間だけ合図を出す等）に直した。

- **過去データで検証（バックテスト）させる**：「同時に合図が出たら何もしない条件で、買いと売りの成績を計算し、損益のグラフも描いて」と頼み、勝率などの数字で効果を確かめる。人が1か月かけていた検証が数分〜数時間でできる。
- **ズル（未来のデータの盗み見）を見張る役のAIを別に作る**：検証プログラムが、本来知りえない「未来のデータ」を見ていないかをチェックする専用のエージェントを用意すると、見かけだけ良い結果にだまされにくい。
- **本番運用は自分のパソコンで動くAIで**：実際の売買に使うなら、ローカルLLM（自分のパソコン内で動くAI）のほうが良い。ネットのAIはサービス側で黙って性能が変わることがあり、判定がぶれるため。
- **自分のやり方を言葉にする**：売買のルールを数式にできるほど言葉にすると、感情的な売買が減る。判断の根拠が「過去の検証結果」になるので、負けても同じルールを淡々と続けられる。

**紹介されたツール** Claude Code（ultrathink）、Gemini API（Gemini 2.5 Flash、Google AI Studio）、TradingView、Python、.envファイル、ローカルLLM

**すぐ実践できること**

- 自分の仕事で「何度も試して良し悪しを確かめたいこと」を1つ選び、AIにまとめて試させる方法がないか考える。
- AIの判定や評価に違和感があったら、「判定に使った基準を一覧にして」と頼んで中身を確認する。

※ 投資には損失のリスクがあります。この回は投資の勧めではなく、AIで検証の仕組みを作る実演です。